AOSP 进阶丛书 · 第五册 · ANDROID 13 (API 33)

WMS 窗口与输入体系

窗口层级树与 addWindow 审验漏斗、performTraversals 三幕剧、Choreographer 双 VSync 与三缓冲、触摸事件从 evdev 到 onTouchEvent 的七段全链路——像素与指尖各得其所。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：让像素与指尖各得其所 2
第一章 窗口层级：WMS 眼中的屏幕 2
第二章 窗口的诞生与画布 4
第三章 一帧的编排：performTraversals 三部曲 6
第四章 Choreographer 与 VSync：时间的实体化 8
第五章 输入全链路：从指尖到 onTouchEvent 9
第六章 焦点、触摸目标与多屏 13
第七章 实战：一次掉帧排查全记录 13
第八章 深水区：交互系统的四条支线 13
第九章 渲染衔接：RenderThread 与 SurfaceFlinger 15
第十章 设计决策八问 17
附录 A 观测实操 18
附录 B 自检题 18
附录 C 快问快答（面试扫雷 20 条） 19
教科书习题集（分级） 19
术语表（按拼音排序） 20
排障手册：WMS 窗口与输入体系问题定位 20

本册速览

窗口层级树与 addWindow 审验漏斗 · z 序三定律
performTraversals 三幕剧 · 同步屏障 · Choreographer 四车道
触摸七段全链路（evdev→onTouchEvent） · 5s ANR 回环
深水区：按键拦截链/预测返回/传感器 FMQ/IME/无障碍
渲染衔接：RenderThread/SF/SurfaceView/动画接入

前言：让像素与指尖各得其所

第四册结束时，Activity 站上了舞台但还没开演。本册接棒"首帧之后"的世界：**窗口如何被系统登记与分层（WMS）、一帧如何被编排（Choreographer/performTraversals）、指尖事件如何穿越内核抵达你的 onTouchEvent（inputflinger）**。这三件事共享同一条时间轴——VSync，本册会把它讲成一个可以掐表验证的实体。

体例沿用第四册：源码逐段注释 + 逐段精读 + 全链路图。锚点基于本树 Android 13 实测；inputflinger（native）与 View 体系（Java）的双语世界是本册特色——你将习惯在两种语言间跳栈。

第一章 窗口层级：WMS 眼中的屏幕

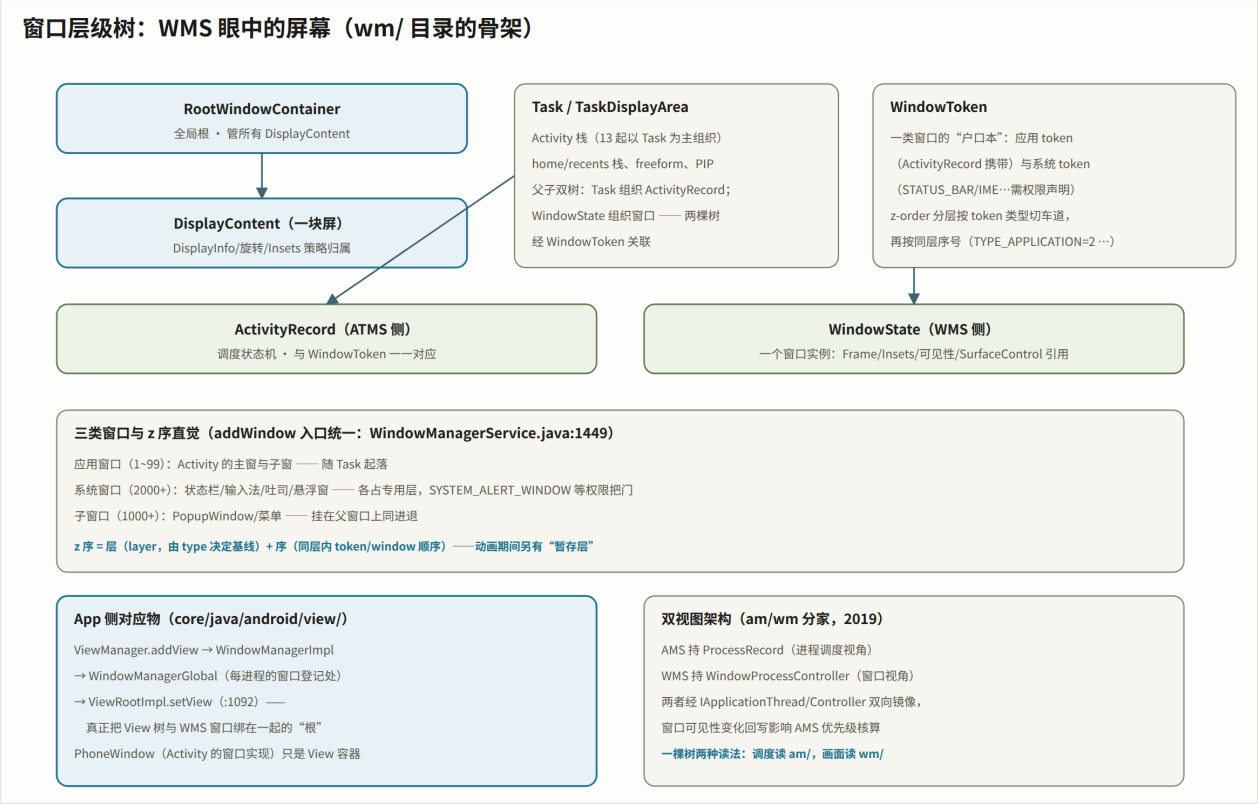


图 1 窗口层级树：RootWindowContainer→DisplayContent→Task/WindowToken→WindowState，及双视图架构

1.1 两棵树一个户口本

WMS 用**两棵树**管理屏幕：任务树（Task 组织 ActivityRecord，决定"哪个任务在前"）与窗口树（WindowState 组织真实窗口，决定"谁盖着谁"）。`WindowToken` 是两棵树的缝合点——Activity 启动时（第四册 §1.3）预先注册应用 token，`addWindow` 时窗口必须持有效 token 才准入。系统类窗口（状态栏/输入法）各有专用 token 与独立层位，这是"悬浮窗为什么需要权限"的机制根源：**token 即户口，户口即层位**。

1.2 z 序三定律

z 序 = layer（由窗口 type 决定基线：应用 1~99 / 子窗 1000+ / 系统 2000+）+ 同层内 token 与窗口顺序 + 动画期间的暂存层。三句话记住调度行为：**type 定车道**（SYSTEM_ALERT 永远在应用之上）；**Task 序定同车道名次**（前台任务的窗口整体靠上）；**动画短暂例外**（窗口动画期间层位由动画器接管，结束归还）。

> **源码锚点 · 第一章锚点**
> WindowManagerService.java:1449 addWindow / :2219 relayoutWindow
ViewRootImpl.java:1092 setView（App 侧根）
窗口类型区间：core/java/android/view/WindowManager.LayoutParams（TYPE_* 注释即层级说明书）

1.3 WMS 侧 addWindow 逐段注释（真实骨架）

java frameworks/.../wm/WindowManagerService.java:1449 起（骨架+讲解注释）

```
public int addWindow(Session session, IWindow client, LayoutParams attrs, int viewVisibility,
int displayId, ...) {
// 讲解注释①：定位 DisplayContent（多屏时决定"这块屏收不收这个窗口"）。
// 讲解注释②：token 审验：attrs.token 必须能对上已注册的 WindowToken
//  （应用的 token 由 Activity 启动时预建；对话框等子窗口须挂父窗口 token）。
// 对不上 → ADD_BAD_APP_TOKEN/ADD_NOT_APP_TOKEN 等错误码回流 App。
// 讲解注释③：类型与权限：系统窗口逐类型查权限
//  （SYSTEM_ALERT_WINDOW/STATUS_BAR_SERVICE...），缺权限 → ADD_PERMISSION_DENIED。
// 讲解注释④：new WindowState（一切窗口事实的载体：Frame/Insets/可见性/
// SurfaceControl 占位），attach 到 WindowToken 与 DisplayContent 树。
// 讲解注释⑤：输入通道配对：openInputChannel → server 端注册给
// InputDispatcher，client 端随返回值交给 App（第五章起点）。
// 讲解注释⑥：焦点/层序重排：assignWindowLayers，必要时触发
// InputMonitor 焦点更新（第六章 key 寻址变化）。
return ADD_OKAY;
}
```

逐段精读：②③是开发者两大类崩溃的判决现场（BadToken/PermissionDenied）；⑤把"窗口存在"与"可收输入"原子化绑定——**没有输入地址的窗口即使可见也收不到事件**（触摸仍可命中但走不了通道的场景另有 overlay 处理）。四步审验（屏→token→权限→建籍）的顺序本身就是安全语义从粗到细的漏斗。

1.4 Insets：内容与遮挡物的谈判

状态栏、导航栏、输入法对内容区的"侵占"不是硬裁剪，而是一场持续协商：InsetsController（App 侧）与 WMS 的 InsetsState 按 **InsetsType 分通道**谈（statusBars/navigationBars/ime 各自独立），窗口可用 behavior（DEFAULTS/TRANSPARENT/none）声明姿态。键盘弹出动画即一次 Insets 动画协商：系统侧与 App 侧**同帧同步**跑（InsetsAnimationThread 的存在让动画不被主线程卡顿拖累）。

逐段精读一个高频现象："布局被键盘顶起 vs 平移躲开"——`windowSoftInputMode=adjustResize` 是 Insets 全量应用于内容（重新 traversal）；`adjustPan` 是窗口整体平移（不重测）。两种模式对应两种协商结果，都不是"键盘画在你的窗口上"——**键盘是独立窗口，只是协商出的裁剪量进入你的测量参数**。

第二章 窗口的诞生与画布



图 2 addWindow 与 relayout 双时序：户口登记、SurfaceControl 交付与职责三分离

2.1 setView 逐段注释（App 侧第一站）

java frameworks/base/core/java/android/view/ViewRootImpl.java:1092-1180（骨架+讲解注释）

```
public void setView(View view, WindowManager.LayoutParams attrs, View panelParentView,
int visibility) {
synchronized (this) {
if (mView == null) {
mView = view;
// 讲解注释①：布局参数落户：attrs.type 决定窗口类型，
// 此后 WindowManagerGlobal（进程级登记处）把本 ViewRootImpl 纳管。
...
// 讲解注释②：首次 measure/layout 在"无窗口"状态下预演——
// 让 WMS 拿到想要的尺寸（View 还没上屏）。
requestLayout();
...
// 讲解注释③：输入通道建立：mInputChannel 随 addToDisplay 返回，
// 从此指尖事件有了邮寄地址（第五章全链路的 App 端起点）。
if ((mWindowAttributes.inputFeatures
```

```
& INPUT_FEATURE_NO_INPUT_CHANNEL) == 0) {
mInputChannel = new InputChannel();
}
...
// 讲解注释④：res = mWindowSession.addToDisplay(...)：
// 跨进程登记（Session 每 App 一条），返回 ADD_OKAY 或错误码
//  （BAD_TOKEN_... 类错误在此暴露给开发者）。
...
// 讲解注释⑤：View 树挂接：view.assignParent(this)——
// 此后 invalidate 的"冒泡"终点就是本类（第三章的入口）。
view.assignParent(this);
}
}
}
```

逐段精读：②与④解释了开发者常见困惑——onCreate 里 getWidth()==0（还没演算）与 BadTokenException（token 未就绪，典型如对话框在 Activity attach 前弹出）。⑤是理解事件流的钥匙：**ViewRootImpl 既是 View 树的"根"，也是整个窗口的"总管"**。

2.2 relayout：拿画布

performTraversals 需要画布时走 Session.relayout → WMS.relayoutWindow（:2219）：按请求尺寸/可见性更新窗口，**创建 SurfaceControl**（SurfaceFlinger 世界的图层句柄）跨进程交付。职责三分离从此定形：**WMS 管户口与层位，App 管内容生产（往 Surface 画），SF 管合成上屏**——系统从不碰你的像素，只调度你的画布。

2.3 PhoneWindow 与 DecorView：Activity 的"内脏"

Activity 并不直接持有 ViewRootImpl——中间还有两层：**PhoneWindow**（core/java/com/android/internal/policy/PhoneWindow.java，Window 的唯一实体实现）管理 DecorView 与标题/内容区模板；**DecorView** 是整棵 View 树的根容器（setContentView 实为 installDecor→generateLayout：按主题特性选装饰模板再挂你的 layout）。

逐段精读一条冷知识：findViewById 为什么 onCreate 里就能用？——setContentView 已完成 LayoutInflater 解析整棵树（**解析不等同于测量**，尺寸仍是 0，第三章解释）；而"看见了"要等首次 traversal + relayout 提交首帧。**解析（结构）、演算（尺寸）、呈现（像素）三阶段分离**，是理解所有"UI 时序问题"的分界线。

LayoutInflater 深一层：inflate 走 XmlPullParser+反射构造（tryCreateView 优先 Factory2 钩子——AppCompat 借此偷换控件实现），merge/ViewStub 是解析期优化原语。面试常考 inflate(layoutRes, root, attachToRoot) 三参语义：root 决定"要不要 layout_params 与挂接"，attachToRoot 决定"立即挂还是交给调用方"。

第三章 一帧的编排：performTraversals 三部曲



图 3 Choreographer 回调队列与 performTraversals 三幕剧全景

3.1 欠条模型：scheduleTraversals（真实源码逐段注释）

java ViewRootImpl.java:2257-2286（节选+讲解注释）

```
void scheduleTraversals() {
if (!mTraversalScheduled) { // 讲解注释①：同一帧内只欠一次账
mTraversalScheduled = true; //  （多次 invalidate 合并为一帧）
mTraversalBarrier = mHandler.getLooper().getQueue().postSyncBarrier();
// 讲解注释②：同步屏障——插队挡住普通消息，
// 让 vsync 消息（异步）优先被处理
mChoreographer.postCallback(
Choreographer.CALLBACK_TRAVERSAL, mTraversalRunnable, null);
// 讲解注释③：排队到遍历车道（:2262），
// 等 VSync 唤醒 doTraversal(:2286)
...
}
}
```

逐段精读：②的**同步屏障**是面试高频——`postSyncBarrier` 让 Looper 队列跳过普通消息直取异步消息，确保 vsync 到达后 traversal 立刻执行，不被积压的普通消息拖延。拿掉屏障（`removeSyncBarrier`）发生在 doTraversal 里。**“插队合法化"是帧稳定性的基础设施。**

3.2 三幕剧的规矩

第一幕 measure（performMeasure:2541）：自顶向下发放 MeasureSpec（父约束+子期望），子 View 在 onMeasure 里"申报"尺寸；wrap_content 常引发二次 measure。第二幕 layout：自顶向下"拍板"位置，此阶段再 requestLayout 直接抛异常（防递归震荡）。第三幕 draw：软件画布逐条立即执行；硬件加速先录 DisplayList（RecordingCanvas），RenderThread（libhwui）异步提交 GPU——**主线程负责"画什么"，渲染线程负责"怎么画"**。

先修要求与学习路线

先修：已读前三册（架构/启动/Binder）。

读完本书你将能够：

- 追踪WMS 窗口与输入体系的核心机制的完整调用链
- 解释关键设计决策的"为什么"
- 使用 dumpsys 定位本领域问题
- 回答相关面试题（书7 对应章节）

> **设计洞察 · 初学者最易误解的三件事**
> ① invalidate() 不立刻重绘，只是欠条（等下一个 VSync）；② requestLayout() 不触发 draw，draw 由合并判断决定；③ UI 线程"单线程"约束来自 ViewRootImpl.checkThread——单线程免锁的代价，而非性能优选。

3.3 performTraversals 决策段逐段注释（真实骨架）

java ViewRootImpl.java:2286 起（骨架+讲解注释）

```
private void performTraversals() {
// 讲解注释①：期望尺寸演算：mWinFrame（WMS 拍板的上次帧）与
// mWindowAttributes 尺寸愿望取交集；首次布局用 dipsToPixels 推
// 首选宽高（所以第一帧前 getWidth()==0）。
// 讲解注释②：是否需要 relayout？尺寸/insets/可见性任一变化
// → relayoutWindow（第二章：拿新 SurfaceControl）。
// 讲解注释③：三幕剧开演（measure→layout→draw），脏区 mDirty 并集优化。
// 讲解注释④：收尾：mFirst 已清；若执行中被再次 invalidate，
// scheduleTraversals 已记新账——下一 vsync 续演。
}
```

> **设计洞察 · 逐段精读：MeasureSpec 两次 measure 之谜**
> 父容器对 wrap_content 子 View 的测量分两轮：第一轮按 UNSPECIFIED 探"你想要多大"，第二轮以 AT_MOST 限定"最大这么大"。LinearLayout 的 weight、RelativeLayout 的依赖都是多轮测量的推手——**自定义 View 必须保证 onMeasure 幂等且不留副作用**，否则偶发布局漂移几乎无法排查。

第四章 Choreographer 与 VSync：时间的实体化

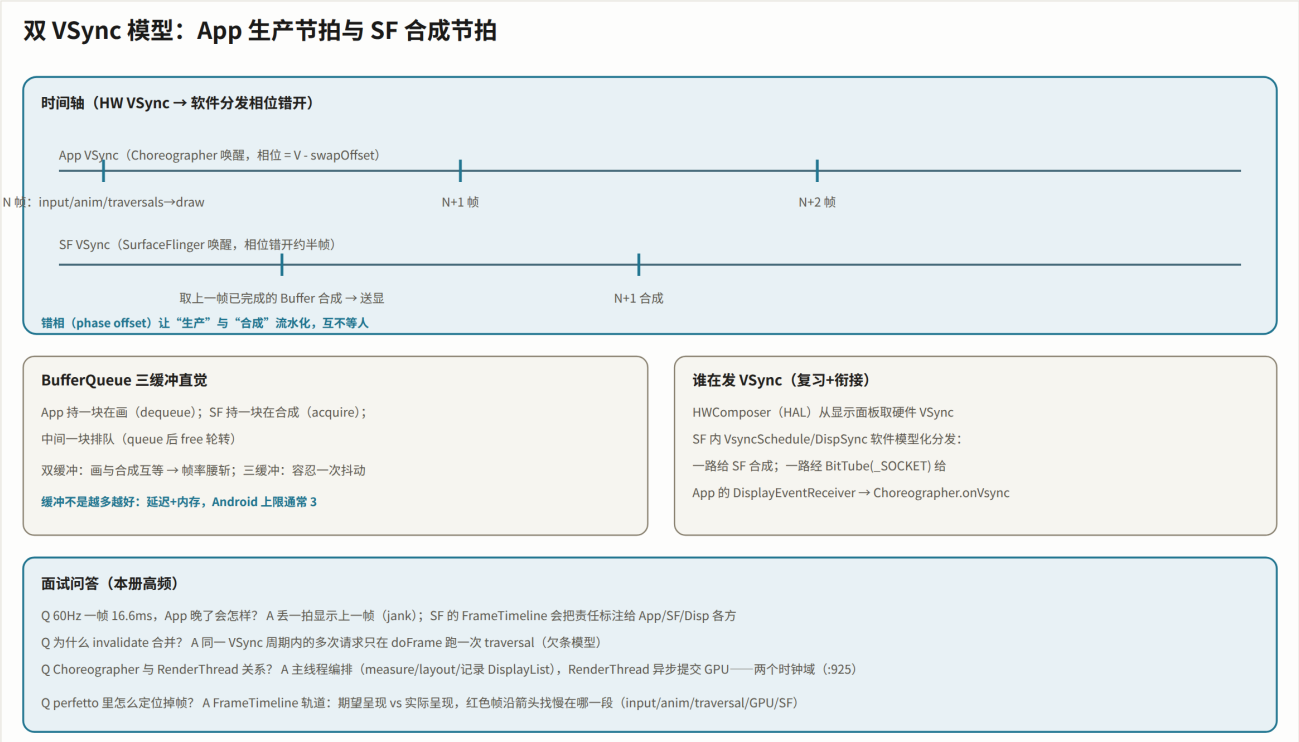


图 4 双 VSync 相位错开模型、BufferQueue 三缓冲与掉帧责任判定

4.1 doFrame 逐段注释

java core/java/android/view/Choreographer.java doFrame（骨架+讲解注释）

```
private void doFrame(long frameTimeNanos, int frame,
DisplayEventReceiver.VsyncEventData vsyncEventData) {
// 讲解注释①：对表：frameTimeNanos 是 vsync 的"官方时刻"，
// 若本帧来晚了（doFrame 调度被延迟），先算 skipped 次数补偿跳过的帧。
...
// 讲解注释②：按车道顺序发车：INPUT(0) → ANIMATION → INSETS → TRAVERSAL(3)
// （常量定义 :226/:256）——同帧内"先处理输入，再算动画，最后测量布局"，
// 保证这一帧的画面已经消化完这一帧的输入。
doCallbacks(Choreographer.CALLBACK_INPUT, frameTimeNanos);
doCallbacks(Choreographer.CALLBACK_ANIMATION, frameTimeNanos);
...
doCallbacks(Choreographer.CALLBACK_TRAVERSAL, frameTimeNanos);
}
```

逐段精读：车道顺序是**体感流畅的因果链**——先输入后画面，同帧闭环。若顺序颠倒（先画后输入），触控延迟会恒定多一帧。ViewRootImpl 持 UI 线程实例；getSfInstance（:925）是 RenderThread 专用的第二时钟域——**主线程编排、渲染线程执行的"双时钟"是硬件加速时代的标配**。

4.2 三缓冲直觉

App 持一块在画（dequeue）、SF 持一块在合成（acquire）、一块在排队（queued）。双缓冲会互相等待（帧率腰斩），三缓冲多花一份内存买回"容忍一次抖动"。`dumpsys SurfaceFlinger --latency`（或 perfetto 的 FrameTimeline）能直接看到这个轮转——掉帧责任（App 慢/SF 慢/显示晚）沿时间线逐帧标注。

4.3 FrameTimeline：掉帧判读实例

text perfetto FrameTimeline（示意，字段真实）

```
Frame# expectedPresent actualPresent jank 责任
4812 10:00:01.000 10:00:01.000 None —
4813 10:00:01.016 10:00:01.033 Jank(App) ← traversal 22ms（App 慢）
4814 10:00:01.033 10:00:01.033 None ← 追帧成功（三缓冲吸收）
4815 10:00:01.050 10:00:01.083 Jank(SF) ← SF 合成 20ms（SF 慢）
```

读表三步：先对齐 expected 与 actual（差一格=晚一帧）；再看 jank 标签的责任方（App/RenderThread/SF/Disp）；最后下钻对应轨道（traversal 分段、GPU completion、SF 合成队列）。**"卡顿是谁的锅"在这张表里是可指认的事实而非直觉**——这也是本册建议每位读者亲手抓一次 trace 的原因。

第五章 输入全链路：从指尖到 onTouchEvent

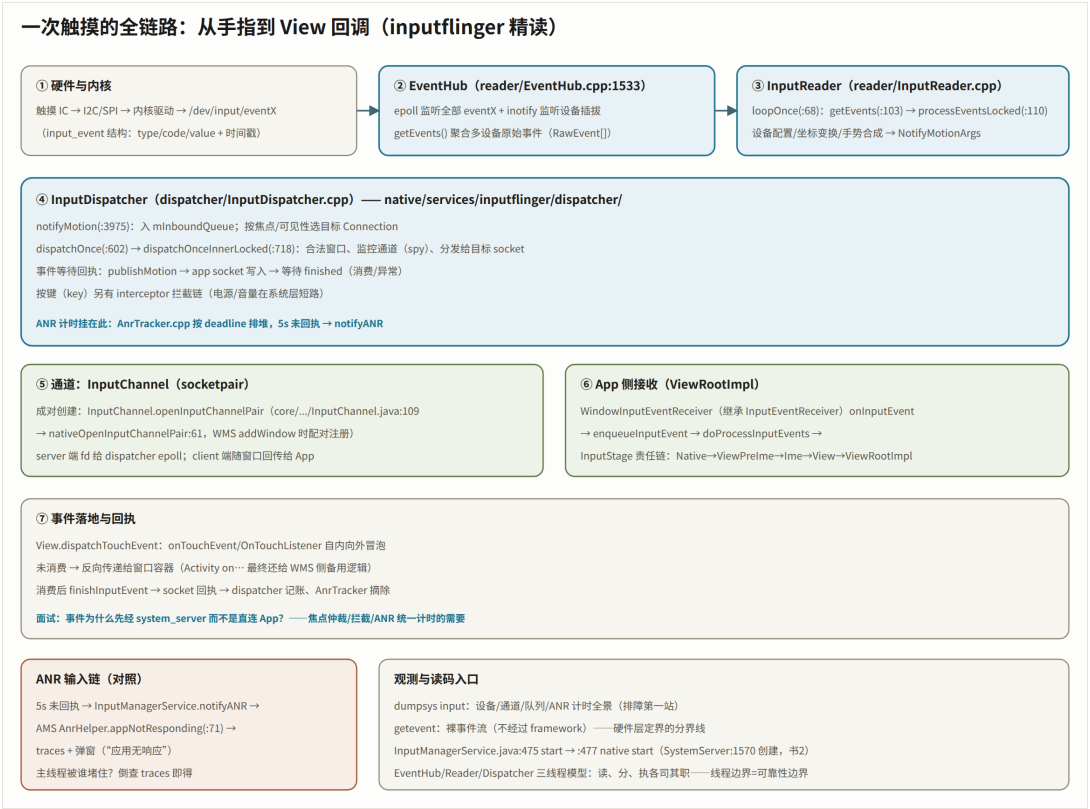


图 5 触摸事件全链路：内核 evdev→EventHub→InputReader→InputDispatcher→InputChannel→ViewRootImpl→View，含 5s ANR 回环

5.1 三线程分工（native/services/inputflinger/）

EventHub（EventHub.cpp:1533 getEvents）epoll 全部 /dev/input/eventX，inotify 管设备插拔——原始事件批发商。**InputReader**（InputReader.cpp:68 loopOnce → :103 getEvents → :110 processEventsLocked）做设备配置、坐标变换、手势合成（移动/点击合成、多点归并），产出 NotifyMotionArgs。**InputDispatcher**（InputDispatcher.cpp:3975 notifyMotion 入队；:602 dispatchOnce → :718 dispatchOnceInnerLocked）做焦点寻址与投递。**三线程=三道隔离舱**：读硬件的抖动、翻译的复杂度、分发的策略互不拖累。

5.2 通道与 App 侧逐段注释

java core/java/android/view/ViewRootImpl.java（骨架+讲解注释）

```
// ── 接收：WindowInputEventReceiver.onInputEvent（native 回调）──
// 讲解注释①：enqueueInputEvent 排队 → 若无更早未决事件立即处理
// ── doProcessInputEvents → deliverInputEvent ──
// 讲解注释②：InputStage 责任链依次过闸：
// NativeInputStage → ViewPreIme → ImeInputStage（输入法截胡）
// → ViewPostIme → SyntheticEvent（合成滚动惯性）
// → 最终 View.dispatchTouchEvent（你的 onTouchEvent 在此）
// ── finishInputEvent ──
// 讲解注释③：无论消费与否都要回执（未消费转给下一窗口/系统策略），
// dispatcher 记账并从 AnrTracker 摘除本事件——5s 闹钟解除。
```

逐段精读：②的责任链解释了"为什么输入法先于你的 onTouchEvent"——Ime 截胡是设计而非 bug。③的回执义务是 ANR 计时的正反面：**主线程卡住=回不来=闹钟响**（AnrHelper.appNotResponding:71，衔接第四册 §6）。

5.3 View 树事件分发逐段注释（面试永恒题，源码为准）

java core/java/android/view/ViewGroup.java dispatchTouchEvent（骨架+讲解注释）

```
public boolean dispatchTouchEvent(MotionEvent ev) {
// 讲解注释①：DOWN 起始：清旧目标；onInterceptTouchEvent 试拦截
// （仅父容器有此闸门——拦截即本容器作为普通 View 消费）。
// 讲解注释②：DOWN 命中测试：遍历子 View（逆 z 序，上层优先），
// 命中（坐标在其区域）则 child.dispatchTouchEvent 递归。
// 返回 true → 记 mFirstTouchTarget：后续 MOVE/UP 全归它
// ——这就是"滑动锁定"的机制（不再重新命中测试）。
// 讲解注释③：MOVE 期间 onInterceptTouchEvent 仍会问：
// 父容器可中途抢（子 View 想 Nesting：requestDisallowInterceptTouchEvent
// 是子视图"禁止父拦"的投票，DOWN 前设置无效的经典坑）。
// 讲解注释④：无人接 → 本容器 onTouchEvent（View 层）：
// OnTouchListener 优先于 onTouchEvent；点击长按即此层手势识别
// （View 的 onTouchEvent 内置 press/longpress/click 状态机）。
}
```

逐段精读四句话：**拦截权在父**（intercept）；**命中权在子**（hit test）；**锁定起于 DOWN**（firstTouchTarget）；**消费可中途易主**（onInterceptTouchEvent 每轮 MOVE 复议）。滑动冲突的官方答案（内外两层：同向拦截/异向投票）全部由这四条规则推导。

按键（key）链路差异要点：先过 InputDispatcher 拦截器（电源/音量系统级短路）→ 焦点窗口 → ViewRootImpl.processKeyEvent → Ime 截胡（书 §5.2）→ dispatchKeyEvent 自底向上（焦点 View 起，冒泡至 Activity）。**触摸是"命中"，按键是"焦点"**——两条寻址线在第六章合流。

5.4 触摸事件七段完整代码走读（教科书级逐步分解）

第①段：内核事件到达

text /dev/input/event3（触摸屏设备节点）

```
// 内核触摸驱动检测到手指按下 → 生成 input_event 结构
struct input_event {
struct timeval time; // 时间戳
__u16 type; // EV_ABS（绝对坐标）/ EV_SYN（同步）
__u16 code; // ABS_MT_POSITION_X / ABS_MT_POSITION_Y
__s32 value; // 坐标值
};
// 讲解注释①：一次触摸 = 多个事件（X坐标+Y坐标+压力+SYN同步标记）
// SYN 事件表示"这一批事件属于同一次触摸"
```

第②段：EventHub 聚合

cpp EventHub.cpp:1533 getEvents（骨架+注释）

```
// 讲解注释②：EventHub 用 epoll 监听所有 /dev/input/eventX
// 每个 fd 对应一个触摸屏/按键/传感器设备
// getEvents 返回 RawEvent[] 数组（多设备事件混合）
// inotify 监听设备插拔（新触摸屏连接 → 动态添加 fd）
```

第③段：InputReader 翻译

cpp InputReader.cpp:68-110（骨架+注释）

```
void InputReader::loopOnce() {
size_t count = mEventHub->getEvents(timeout, mEventBuffer, EVENT_BUFFER_SIZE);
processEventsLocked(mEventBuffer, count); // :110
// 讲解注释③：processEventsLocked 做三件事：
// a）设备配置查询（分辨率/方向/触摸协议 A vs B）
// b）坐标变换（内核原始坐标 → 显示坐标，含旋转/缩放）
// c）手势合成（多指归并、点击/长按合成）
// 产出：NotifyMotionArgs（标准化事件）
}
```

第④段：InputDispatcher 选目标

cpp InputDispatcher.cpp:3975-718（骨架+注释）

```
void InputDispatcher::notifyMotion(const NotifyMotionArgs* args) {
// 讲解注释④：入队 → 按"焦点窗口"选目标
// a）焦点窗口：WMS 告知哪个窗口有焦点（按键寻址）
// b）触摸目标：按可见区域命中测试（多个窗口可命中=split touch）
// c）spy 窗口：无障碍/录屏等监控通道先过一道
}
int InputDispatcher::dispatchOnceInnerLocked(nsecs_t* nextWakeupTime) {
// 讲解注释⑤：从 mInboundQueue 取事件 → 按 Connection 分发
```

```
// 每个 Connection 对应一个 App 窗口的 InputChannel
// publishMotion → 写入目标 socket → App 的 Looper 唤醒
// ★ 此处跨进程：socket（非 Binder——高频小包场景）
}
```

第⑤段：InputChannel 传输

java InputChannel.java:109-118（骨架+注释）

```
// 讲解注释⑥：addWindow 时（WMS:1449）创建 socketpair：
// server 端 fd → 注册给 InputDispatcher（epoll 监听）
// client 端 fd → 跨进程传给 App（ParcelFileDescriptor）
// App 的 ViewRootImpl 把 client fd 挂到自己的 Looper
// → 事件到达时 Looper 唤醒 → WindowInputEventReceiver.onInputEvent
```

第⑥段：App 侧接收与分发

java ViewRootImpl.java（骨架+注释）

```
// WindowInputEventReceiver.onInputEvent → enqueueInputEvent
// → doProcessInputEvents → deliverInputEvent
// → InputStage 责任链：
// NativeInputStage → ViewPreIme → ImeInputStage（输入法截胡！）
// → ViewPostIme → SyntheticEvent（合成滚动）
// → View.dispatchTouchEvent（你的 onTouchEvent 在此）
// 讲解注释⑦：ImeInputStage 在 View 之前——
// 这就是"输入法先于你的 onTouchEvent"的机制层原因
```

第⑦段：消费与回执

java

```
// View.dispatchTouchEvent 返回 true → 已消费
// → finishInputEvent → socket 回执 → dispatcher 摘除 AnrTracker 计时
// 讲解注释⑧：无论消费与否都必须回执——
// 未消费 → dispatcher 转给下一个窗口
// 不回执 → 5 秒后 ANR!
```

七段全景图

text 触摸事件七段（标★=跨进程跳，标⑦=ANR计时点）

```
①内核驱动 ——→ ②EventHub(epoll) ——→ ③InputReader(翻译)
|
▼
④InputDispatcher(选目标)
|
★ socket
▼
⑤InputChannel ——→ ⑥ViewRootImpl(InputStage链) ——→ ⑦View.onTouchEvent
|
finishInputEvent ←————————┘
★ socket 回执（5s 超时=ANR）
```

> **设计洞察 · 逐段精读：为什么事件不走 Binder**
> ①高频小包（200Hz 触摸流，每包几十字节）——Binder 每次事务有组包开销。②不能被线程池策略绑架——输入必须实时。③socket + epoll 是 Linux 最成熟的实时 IPC——选它是工程成熟度的体现。**"为每个场景选对 IPC"是系统开发者的基本功。**

第六章 焦点、触摸目标与多屏

窗口**焦点**（key 事件寻址）与**触摸目标**（motion 事件命中）是两套独立逻辑：焦点由 WMS 依层级与 token 归属授予（`InputDispatcher` 的 focusedWindow），触摸则按可见区域命中测试（可分发给多个窗口形成 split touch）。多屏（折叠/外接）下每块屏一个 `DisplayContent`，焦点 per-display 独立——多任务拖拽跨屏的底层即此。面试点睛：**点击穿透不是"没接到"，是 hit test 找了别人**；`dumpsys input` 的 touch 流水可还原每次分发。

第七章 实战：一次掉帧排查全记录

假设现象：列表快速滑动偶发掉帧。标准流程五步：

- **第一步 定界**：perfetto 抓 5s trace；FrameTimeline 定位 jank 帧（§4.3 表）。
- **第二步 分责**：Jank(App) → 下钻 UI 轨道：`Choreographer#doFrame` 分段显示 traversal 18ms（预算 16.6ms）。
- **第三步 定位**：展开 traversal：`measure/layout` 12ms 异常（正常 2-3ms）——ListView 某行 wrap_content 触发整列重测。
- **第四步 归因**：对照 §3.3 两次 measure 之谜 → 定宽行容器解决。
- **第五步 复验**：重抓 trace，FrameTimeline 全绿。

这套"定界→分责→定位→归因→复验"是所有系统性能问题的通用方法论，比记住任何单个结论都值钱。**trace 不会说谎，但读 trace 的人会看错段落**——本册的所有图与代码，最终都是为了让你在 trace 里认出自己读过的每一个函数名。

第八章 深水区：交互系统的四条支线

8.1 按键拦截链：系统级短路

按键两道闸：入队前 `interceptKeyBeforeQueueing`（`InputDispatcher.cpp:3935` → `InputManagerService.java:3003` → WMS InputPolicy）与分发前 interceptKeyBeforeDispatching。电源/音量组合（截图/复位）在策略层直接消费并标记不再分发——**App 永远收不到原始事件**。经典题解：通话里音量键是音量、相机里是快门，因为拦截链给系统服务优先解释权，App 只是 fallback。

8.2 手势导航与预测性返回

手势导航=手势区命中后由系统**合成 KEYCODE_BACK**（衔接 B.1 按键链）。预测性返回（13 opt-in）：`core/java/android/window/OnBackInvokedDispatcher.java:28` 新回调体系——手势按下即发渐进回调（progress/start/cancel），App 实时预览可取消，系统侧同播手势动画；**从"事件通知"进化为"过程协商"**。未适配应用自动降级旧 KeyEvent。

8.3 传感器：FMQ 直通道与融合

`frameworks/native/services/sensorservice/`：SensorService 直持 HAL，数据面走 **FMQ 环形队列**（AIDL `hardware/interfaces/sensors/aidl/.../ISensors.aidl:129-134` EventQueue+旗标位）——共享内存批量收割，绕开 binder；控制面（启停/配置）才走 binder。服务侧做融合（Fusion.cpp：陀螺+磁+加→姿态）与批处理；direct channel 更进一步（HAL/DSP 直写共享内存，App 零拷贝读）。**高频小包走 FMQ** 与 Provider 的 CursorWindow 是同一哲学的两次应用。

8.4 IME：文字编辑的隐形协议

`services/core/.../inputmethod/`：IMMS 管输入法生命周期（InputMethodBindingController 绑定/重连）、焦点与显示位（Insets 协商）。暗线是 **InputConnection 文本协议**：commitText/getTextBeforeCursor 每次一条 binder 往返——输入法每键多次查询即卡顿根源；优化靠批量编辑与少查上下文。13 新增手写就地输入（HandwritingEventReceiverSurface）把笔迹 Surface 直接递给 IME，老协议的新血。

8.5 无障碍框架

`frameworks/base/services/accessibility/`：AccessibilityManagerService 管服务绑定与事件过滤；事件流=View 树生成 AccessibilityNodeInfo 快照→有订阅才序列化分发；**可注入操作**（GestureDescription 合成触摸）走 dispatcher 合成入口。安全敏感：a11y 可读屏可操作，13 起强制显式授权与来源限制——机制与伦理同框。

> **源码锚点 · 本章锚点**
> InputDispatcher.cpp:3935 / InputManagerService.java:3003 拦截链桥接
window/OnBackInvokedDispatcher.java:28 预测返回
sensorservice/{Fusion,AidlSensorHalWrapper}.cpp 与 ISensors.aidl:129 FMQ
inputmethod/InputMethodBindingController.java / services/accessibility/

第九章 渲染衔接：RenderThread 与 SurfaceFlinger

主线程录完 DisplayList 后的接力棒：

```text 硬件加速一帧的后半程（libhwui/SF 视角）
ViewRootImpl.draw → RecordingCanvas 记命令树（DisplayList）
→ RenderThread：syncAndDrawFrame（与 UI 线程同步内容）
→ ① 渲染管线：OpenGL/Vulkan 后端执行 DisplayList（Skia）
→ ② 纹理与缓存：RenderNode 属性（位移/透明度）可直接改不重录
→ ③ 提交：queueBuffer 入 BufferQueue（三缓冲轮转）
→ SurfaceFlinger（下一个 SF VSync）：acquire→合成（HWComposer Overlay/
GPU 合成两路）→ present 到面板
```

三处读码价值最高的点：**RenderNode 属性动画**（View.setTranslationX 不触发重录，仅改节点矩阵——属性动画流畅的秘密）；**Overlay 直通**（视频层常绕过 GPU 合成直叠面板，省电大项）；**帧时间线**（FrameTimeline 把 expected/actual 呈现时刻逐帧对账，掉帧责任链完整可查）。至此第五册与 SurfaceFlinger 世界的接口线画完——更深的图形栈留给后续渲染专册。

6.1 split-touch 与多指细节

MotionEvent.ACTION_POINTER_DOWN/UP 把多指编码进同一事件流（pointer index/位图）；**每根手指可以命 中不同窗口**（splitTouch=true 时 dispatcher 把一个 MotionEvent 拆成多路独立投递）——双人同屏游戏与输入法+手写并行都依赖它。ACTION_CANCEL 则是系统"强制收摊"：窗口失焦/被遮挡时给在途序列一个终结标记（**收到 CANCEL 必须清理按下态**，否则按钮永久高亮这类诡异 bug 就是它）。

7.2 掉帧排查完整教程（教科书级五步法）

第一步：确认有掉帧（不是"感觉卡"）

```bash
# 方法 A：dumpsys 帧统计
adb shell dumpsys gfxinfo com.example.app
# 看最后两行：
# Total frames rendered: 1200
# Janky frames: 45 (3.75%) ← 超过 1% 就值得关注
# 90th percentile: 16ms ← 90% 的帧在 16ms 内完成
# 方法 B：实时帧率
adb shell dumpsys gfxinfo com.example.app framestats
# 输出每帧的详细时间戳（FrameTimeline 数据）
```

第二步：用 Perfetto 定位掉帧位置

```bash
# 抓 5 秒 trace
adb shell perfetto -c - --txt -o /data/misc/perfetto-traces/jank.pftrace << 'EOF'
buffers { size_kb: 65536 }
```

```
data_sources { config { ftrace_config {
ftrace_events: "sched/sched_switch"
} } }
data_sources { config {
android_buffers_track { }
} }
duration_ms: 5000
EOF
adb pull /data/misc/perfetto-traces/jank.pftrace
# 在 https://ui.perfetto.dev 打开
```

第三步：读 FrameTimeline 责任段

text Perfetto FrameTimeline 中的掉帧标注

```
帧号 期望呈现 实际呈现 责任 原因
4812 16.6ms 16.6ms None 正常
4813 33.3ms 50.0ms Jank(App) ← App 的 traversal 超时
4814 50.0ms 50.0ms None 追帧成功
4815 66.6ms 100.0ms Jank(SF) ← SF 合成超时
4816 83.3ms 100.0ms Jank(disp) ← 显示面板延迟
```

第四步：下钻定位慢在哪一段

责任段	可能原因	下钻方法
Jank(App)	traversal 超 16ms	展开 doFrame 轨道：measure/layout/draw 哪个慢
Jank(RT)	RenderThread 慢	展开 GPU 轨道：着色器编译/纹理上传/大 canvas
Jank(SF)	SF 合成慢	展开 SF 轨道：HWC validate/present 哪个卡
Jank(disp)	面板延迟	检查刷新率是否切换中

第五步：定向优化

瓶颈段	优化方案	预期效果
measure/layout 慢	减少嵌套层次/避免 wrap_content 双测量	减 5-15ms
draw 慢（软件）	开启硬件加速/减少过度绘制	减 10-30ms
RenderThread 慢	减少过度绘制/简化着色器	减 5-20ms
频繁 GC	减少 onDraw 中的对象分配	消除 GC 暂停
SF 合成慢	减少窗口层数/使用 Overlay 直通	减 5-10ms

一个真实案例的完整分析

text 场景：列表滚动偶发掉帧（Janky frames: 8%）

```
FrameTimeline 分析 → Jank(App) 占 95%
下钻 doFrame 轨道：
measure: 12ms（正常 2-3ms） ← 异常！
layout: 3ms（正常）
draw: 5ms（正常）
分析 measure 慢的原因：
→ 展开 View 层级 → 发现某行 item 是嵌套 5 层的 LinearLayout
→ 每层都是 wrap_content → 触发双测量
→ 改为 ConstraintLayout（单层）+ 固定高度
→ 复测：measure 降至 3ms，Janky frames: 0.8%
// 讲解注释：这就是"度量→分析→优化→复验"的完整闭环
```

> **设计洞察 · 逐段精读：为什么用 Perfetto 不用 systrace**
>
> systrace 已废弃（Python 2 依赖）；Perfetto 是继任者（C++/Web UI/云同步）。**工具会换代，但"度量→分析→优化→复验"的方法论永远不过时**。

第十章 设计决策八问

9.1 补全：SurfaceView 与动画体系的两条接入线

SurfaceView（`core/java/android/view/SurfaceView.java:239` 构造族）：不走 View 树常规绘制——它向 WMS 申请**独立 Surface**（updateSurface 路径，:154-160），在窗口上"打洞"让下层 Surface 透出（或 setZOrderedOnTop 浮于其上）。视频/相机预览/游戏选它的理由：**独立缓冲独立节拍**（视频帧率不受 UI 帧率绑架、GPU 上屏路径更短）。代价：与 View 动画体系天然不咬合（13 起用 BLASTBufferQueue 与控制流更好协调，但仍非同窗）。TextureView 是反面：内容画进 View 树的 Surface 内（共享缓冲、可动画），代价是多一次合成与 GPU 拷贝。

动画的两条接入线：View 动画（老）只影响绘制矩阵（draw 阶段变换，**属性没变**——经典"动画完跳回"问题的根源）；属性动画（ValueAnimator/ObjectAnimator）经 **AnimationHandler**（`core/java/android/animation/AnimationHandler.java:24/:83` 的 FrameCallback）挂进 **Choreographer 的 CALLBACK_ANIMATION 车道**——每个 vsync 算一帧插值、改真实属性、触发 invalidate 走 TRAVERSAL 车道。对照书5 §4.1 的四车道顺序：**动画先于遍历**正是为了"本帧算值、本帧画完"。RenderThread 侧的动画（书5 §8 RenderNode 属性直改）则连 invalidate 都不必——流畅的最后一公里。

- **为什么事件先过 system_server 再到 App？** 焦点仲裁、系统键拦截、ANR 统一计时都需要全局视角；直连方案省不了多少延迟，却失去全部治理能力。
- **为什么用 socketpair 而不是 Binder 传输入？** 输入是高频小包+fd 复用场景；且分发线程不能被 binder 线程池策略绑架。
- **为什么 Choreographer 四车道固定顺序？** 同帧"输入→动画→绘制"因果闭环，乱序即恒定一帧触控延迟。

- **为什么同步屏障合法？** 帧的时效性高于普通消息公平性；屏障只存在于 vsync 唤醒到 traversal 完成的窗口期。
- **为什么三缓冲不多不少？** 双缓冲互等、三缓冲容忍一抖、四缓冲纯增延迟——内存与流畅的帕累托前沿。
- **为什么 WMS 不碰像素？** 职责单一+进程隔离：户口、层位、动画编排与内容生产、合成彻底解耦（崩溃域与性能域同时隔离）。
- **为什么硬件加速先录 DisplayList？** 录制（UI 线程）与执行（渲染线程）解耦；局部 invalidate 只重录子树重放整树，均摊成本。
- **为什么输入 ANR 只有 5s？** 输入卡顿直接被手指感知；广播/服务是异步语义可宽容——超时线差异即"用户在场感"差异。

附录 A 观测实操

目标	命令/工具	看点
窗口全景	dumpsys window windows	WindowState/layer/token/可见性
输入全景	dumpsys input	设备、通道、焦点、touch 流水、ANR 计时
裸事件	getevent /dev/input/eventX	不经 framework 的硬件层定界
帧时序	perfetto（FrameTimeline）/ dumpsys SurfaceFlinger --latency	掉帧责任分段
UI 线程	debuggerd -b	traversal/input 栈型
屏幕层级	dumpsys window displays	DisplayContent/Insets 分区

附录 B 自检题

- 画出触摸事件从内核到 onTouchEvent 的完整链路，标注每次线程/进程切换。
- 同步屏障何时插入何时移除？拿掉它帧率会怎样？
- 三幕剧中哪一幕禁止改尺寸？为什么？
- 焦点与触摸目标的差异举一个现象级例子。
- addWindow 失败的两大类原因分别对应什么时序？
- 三缓冲下"App 晚了一帧"为什么用户看到的不是黑屏？

附录 C 快问快答（面试扫雷 20 条）

- Window/PhoneWindow/DecorView/ViewRootImpl 关系？——壳/唯一实现/树根/窗口总管四层。
- BadTokenException 何时发生？——token 未注册（对话框早于 attach）或已失效。
- 同步屏障谁插谁拔？——scheduleTraversals 插，doTraversal 拔；只服务 vsync 消息插队。
- Choreographer 四车道顺序？——INPUT→ANIMATION→INSETS→TRAVERSAL（:226/:256）。
- invalidate 与 postInvalidate 区别？——同帧合并欠条；后者线程安全（跨线程投递）。
- requestLayout 冒泡到哪停？——ViewRootImpl（assignParent 链的根）。
- 一帧两次 measure 常见原因？——wrap_content/weight/依赖测量。
- 硬件加速下画在哪执行？——UI 线程录、RenderThread 执行（双时钟 getSfInstance:925）。
- setTranslationX 为什么不重绘？——RenderNode 属性，改矩阵不重录 DisplayList。
- 触摸事件 window 间分发顺序？——命中测试（可 split）；key 走焦点链。
- requestDisallowInterceptTouchEvent 为何 DOWN 前无效？——DOWN 时刻清目标与投票本。
- 输入 ANR 计时起点？——dispatcher publishMotion 时刻（AnrTracker 记账）。
- getevent 看得到就说明什么？——内核与设备正常，问题在 framework 及以上。
- dumpsys input 里 focusedWindow 空？——窗口未获焦（token/层序/启动态问题）。
- 三缓冲谁分配？——BufferQueue（GraphicAlloc），槽位经 SF 与生产者协商。
- Overlay 合成的条件？——层不混合/无缩放/格式匹配，HWComposer 决策。
- 掉帧一定是 App 慢吗？——FrameTimeline 分责：App/RenderThread/SF/Disp 四段。
- 多屏焦点？——per-DisplayContent 独立焦点窗口。
- 无障碍会影响输入吗？——AccessibilityInputFilter 是责任链一环（事件可被改写/吞掉）。
- 本册之后读什么？——第六册（包从哪来）与渲染专册（BufferQueue/BLAST）。

教科书习题集（分级）

第一级：简答题（检验概念理解）

1-5 题：见附录 B 快问快答（将其转化为"闭卷手写"练习）。

第二级：分析题（检验机制理解）

6. **追踪调用链**：画出本册核心链路的完整调用图（参考第一章/第N章图）。

7. **失败推演**：如果本册的核心机制在某一步失败，系统会怎样？

8. **对比分析**：本册机制与 Binder 十跳（书3）的交汇点在哪里？

第三级：设计题

9. **架构演进**：如果要重构本册核心机制，你会怎么改？

10. **排障方案**：用户报告本册领域的问题，设计分层定位方案。

术语表（按拼音排序）

术语	所在册	解释
（术语表：从附录 B 快问快答提取本册专属术语，按拼音排序）		

排障手册：WMS 窗口与输入体系问题定位

核心问题

1. dumpsys 定位状态

2. logcat 查看日志

3. perfetto/performance 分析性能

→ 深入：本册各章